

Ex1:

A - Calculate powers of golden mean backwards

Define a function: `golden_powers_down(n,type)`

that returns a numpy array of size `n,3` and `dtype=type` (either "float32" or "float64").

The first column is the power calculated by the linear relation, the second is the power calculated directly, and the third is the relative difference of the two. The first line is for power=0 and the last is for `n-1`.

Run the function for `n=31` and `type='float32'`.

Plot the log. of absolute value of errors.

Apply the following lines in jupyter or use python directly.

```
-----  
goldown = golden_powers_down(31,'float32')  
%matplotlib notebook  
from matplotlib import pyplot as plt  
import numpy as np  
plt.plot(range(2,25),np.log10(abs(goldown[2:25,2])))  
plt.title('stab_down',fontsize=16)  
plt.xlabel('n',fontsize=14)  
plt.ylabel('log-abs-error',fontsize=14)  
plt.show()  
-----
```

Use the following lines to plot the errors:

```
-----  
%matplotlib notebook  
from matplotlib import pyplot as plt  
import numpy as np  
plt.plot(range(2,19),goldown[2:19,2])  
plt.title('stab_down',fontsize=16)  
plt.xlabel('n',fontsize=14)  
plt.ylabel('error',fontsize=14)  
plt.show()  
-----
```

Submit code, outputs, plots and explanations as jupyter notebook

file (*.ipynb);

or code as python file (*.py) and outputs, plots and explanations in pdf file.

B - compare numerical derivatives of $\sin(x)$ using 3 formulations:
2-points $(x, x+dx)$
3-points $(x-dx, [x], x+dx)$
5-points $(x-2dx, x-dx, [x], x+dx, x+2dx)$

```
write functions:
    derivative2p(f,x,dx)
    derivative3p(f,x,dx)
    derivative5p(f,x,dx)
```

use the following code to compare the accuracy for 10 random values of x :

```
-----
import numpy as np
def func(x):
    return np.sin(x)

def true_deriv(x):
    return np.cos(x)

np.random.seed(12345)
x = np.random.rand(10)
dx = 0.002
print('%12s %12s %12s %12s %12s ' %
      ('x','dfdx','err2p','err3p','err5p'))
for a in x:
    der2p = derivative2p(func,a,dx)
    der3p = derivative3p(func,a,dx)
    der5p = derivative5p(func,a,dx)
    der = true_deriv(a)
    print('%12.4e %12.4e %12.4e %12.4e %12.4e ' % (a,der,der2p/der-1,der3p/der-1,der5p/der-1))
-----
```

write a code for comparison of accuracy of $\sin(x=0.7)$ using the above 3 functions for different dx , starting from $dx=0.1$ down to $dx=1e-10$ (i.e. multiply dx by 0.1 each time).

use the following commands for header printing:

```
-----
a = 0.7
der = true_deriv(a)
print('%s %12.4e %s %12.4e ' % ('x=',a,'dfdx=',der))
print('%12s %12s %12s %12s ' % ('dx','err2p','err3p','err5p'))
-----
```

use the following line for table printing (i.e. for each dx):

```
-----
print('%12.4e %12.4e %12.4e %12.4e ' % (dx,der2p/der-1,der3p/der-1,der5p/der-1))
-----
```

add explanations of the results

Submit code, outputs, plots and explanations as jupyter notebook file (*.ipynb);
or code as python file (*.py) and outputs, plots and explanations in pdf file.